

IPSNS for Minimum Weighted Feedback Arc Set on Sparse Digraphs

Soroush Vahidi [ORCID](#)^{1*}

^{1*}Department of Computer Science, New Jersey Institute of Technology,
Newark, 07102, NJ, USA.

Corresponding author(s). E-mail(s): sv96@njit.edu;

Abstract

The minimum weighted feedback arc set problem seeks a minimum-weight arc set whose removal makes a directed graph acyclic, or equivalently a vertex ordering minimizing backward-arc weight. We study sparse nonnegative weighted digraphs, where exact methods do not scale broadly and dense matrix-based ordering heuristics are not always structurally aligned.

We introduce IPSNS (incumbent-protected SCC neighborhood search), an SCC-local destroy-and-repair heuristic centered on strongly connected components (SCCs) that still contribute backward weight after constructive seeding. IPSNS starts from the better of two attributed seeds: a local-ratio topological add-back construction derived from prior local-ratio work and a WMSF-style weighted minimal/stable construction adapted from Cavallaro and Cutello. It scores SCCs by incumbent backward contribution, perturbs fixed SCC neighborhoods, repairs them with SCC-restricted local-ratio reduction and heavy-first add-back, and accepts a candidate only under strict global improvement. The supporting analysis establishes feasibility, add-back minimality, and incumbent non-worsening, but no new approximation ratio.

On the primary sparse benchmark, IPSNS attains the minimum observed backward weight among the evaluated methods on 96 of 97 standard nonnegative instances. On the 57-instance exact-validation subset, it matches the exact dynamic-programming optimum on 56 instances. In a repeated-run comparison on the 93-instance common sparse subset, per-instance medians over 20 runs give 38 wins, 55 ties, and 0 losses versus the evaluated DRMacIver/FAS executable. A dense LOLIB transfer test marks the scope boundary: matrix-based ordering methods remain preferable on complete weighted ordering instances.

Keywords: Minimum weighted feedback arc set, Combinatorial optimization, Local-ratio algorithm, Strongly connected components, Heuristic search, Algorithm engineering

1 Introduction

The minimum weighted feedback arc set (MWFAS) problem asks for a minimum-weight set of arcs whose removal makes a directed graph acyclic; equivalently, it asks for a vertex ordering with minimum total backward-arc weight. The problem is NP-hard [1] and arises whenever directed pairwise relations contain inconsistent cycles, including precedence aggregation, dependency repair, scheduling, and ranking from directed evidence [2, 3]. In weighted settings, not all violations are interchangeable: reversing a high-weight precedence relation is more costly than reversing a weak one, so useful heuristics must react to both graph structure and arc-weight asymmetry.

This paper focuses on *sparse nonnegative weighted digraphs*. That regime differs structurally from dense linear-ordering or tournament instances. In many sparse graphs, large regions are already acyclic or only weakly cyclic, while the hard part of the objective is concentrated in a smaller set of strongly connected components (SCCs). Dense complete ordering instances behave differently: every ordered pair carries weight, the cyclic core effectively spans the whole graph, and matrix-based pairwise-ordering heuristics are naturally aligned with the input model. A method that is strong on sparse digraphs therefore need not be strongest on dense ordering benchmarks.

This creates a practical gap. Exact formulations provide certification but do not scale broadly enough to serve as the main tool on the sparse benchmark studied here [4]. Classical greedy heuristics remain fast and interpretable, yet they may spend effort too globally or may not exploit the fact that residual backward weight is often localized after a strong constructive pass. Earlier local-ratio and weighted minimal/stable constructions provide useful seeds, but they leave open whether a *protected* SCC-local refinement stage can improve solutions consistently without ever returning a worse answer than the best available seed.

The central contribution is *incumbent-protected SCC neighborhood search* (IPSNS). IPSNS begins from the better of two attributed seeds, scores nontrivial SCCs by their current backward-weight contribution, and applies SCC-local destroy-and-repair moves inside fixed original-graph neighborhoods. The destroy phase reactivates heavy removed internal arcs and removes light active internal arcs. The repair phase reruns local-ratio reduction and heavy-first add-back only inside the selected SCC neighborhood. A candidate is accepted only if the repaired global ordering is feasible and strictly improves backward weight; otherwise the incumbent is restored exactly. IPSNS is therefore stochastic in neighborhood choice but monotone in returned objective value.

The two seeds are supporting components rather than coequal contributions. LR-TA (local-ratio topological add-back) is an engineered sparse-graph seed derived from the directed local-ratio lineage of Demetrescu and Finocchi [5] and connected to earlier author work on ranking-oriented MWFAS [6]. The WMSF-style seed is an adapted implementation of the weighted minimal/stable feedback arc set line of Cavallaro and Cutello [7]. The present paper does not claim those foundations as new; it uses them as complementary entry points whose better incumbent IPSNS then protects and selectively improves.

The empirical message is intentionally narrow. On the primary sparse comparison, IPSNS attains the minimum observed backward weight among the evaluated methods on 96 of 97 standard nonnegative instances. On the 57-instance exact-validation subset, it matches the dynamic-programming optimum on 56 instances. On the 93-instance common sparse subset, repeated-run per-instance medians over 20 runs yield 38 wins, 55 ties, and 0 losses for IPSNS against the evaluated DRMacIver/FAS executable. A dense LOLIB transfer study marks the regime boundary: matrix-based ordering methods remain preferable on complete weighted ordering instances. The paper’s claim is therefore that protected SCC-local refinement is an effective sparse-graph algorithm-engineering strategy among the evaluated methods, not that it solves every MWFAS regime equally well.

The contributions are:

- **Algorithmic contribution.** We introduce IPSNS, an incumbent-protected SCC-local destroy-and-repair heuristic for sparse nonnegative weighted digraphs. The contribution lies in the problem-specific integration of fixed SCC neighborhoods, contribution-based SCC selection, two-sided perturbation, SCC-restricted repair, and strict global-improvement acceptance.
- **Integrated framework.** We evaluate IPSNS together with two explicitly attributed seeds: an engineered LR-TA seed derived from prior local-ratio work and a WMSF-style seed adapted from Cavallaro and Cutello’s weighted minimal/stable pipeline.
- **Supporting analysis.** We formalize implementation-level properties for seed feasibility, one-pass add-back minimality, and IPSNS incumbent non-worsening, together with concise complexity summaries. These are supporting guarantees for the implemented framework rather than new approximation results.
- **Computational evidence.** We report sparse benchmark comparisons, exact and MIP validation, ablation and holdout-supported parameter checks, repeated-run robustness against an external executable with run-to-run variation, and a dense LOLIB scope-boundary study.

The remainder of the paper positions the work, defines the objective, presents the framework, summarizes the experimental design, reports the claim-bearing results, discusses the sparse and dense regimes, and concludes.

2 Related work

2.1 Feedback arc set algorithms

Minimum feedback arc set sits between graph-theoretic cycle breaking and ordering optimization. Classical formulations treat it as deleting a minimum-weight arc set to obtain an acyclic digraph or, equivalently, as finding an ordering of minimum backward weight [2, 8]. The decision version is NP-hard [1], and exact methods remain important mainly as certification tools on restricted instance classes. Modern exact formulations connect MWFAS to broader combinatorial-optimization machinery [4], but they do not eliminate the practical need for heuristics on larger sparse graphs.

Approximation and reduction-based work provides methodological context rather than the main claim of this paper. The general local-ratio framework of [9] explains why iterative weight reductions can preserve improvement structure in weighted optimization. For directed feedback problems, Demetrescu and Finocchi [5] show how cycle-based local-ratio reductions can be used algorithmically; that work is the main conceptual ancestor of the LR-TA seed used here. Earlier directed-feedback approximation results [10] and more recent localization-oriented engineering work [11] place the present heuristic study within a broader feedback-set literature.

Classical heuristic baselines remain relevant because many practical implementations still inherit their design logic. Greedy score-based ordering rules in the Eades family [12, 13] are fast and interpretable, but they act mainly through global score differences rather than targeted treatment of residual cyclic subgraphs. Comparative studies such as [14] show that SCC-aware decomposition often improves over whole-graph greedy ranking, which is directly relevant to the local repair strategy of IPSNS. At larger scales, deployment-oriented engineering becomes central [15].

2.2 Ordering and ranking formulations

The ordering interpretation of MWFAS connects the problem to ranking and linear ordering. In dense pairwise-comparison settings, the natural input is often a non-negative matrix W whose entry W_{ij} expresses evidence that item i should precede item j , and the objective is to maximize total forward weight or minimize total backward weight [3, 16]. That viewpoint is especially natural for complete tournaments and dense linear-ordering benchmarks. In sparse digraphs, many ordered pairs are absent altogether, so the problem is more naturally expressed as cycle breaking in an incomplete directed graph.

This distinction matters for baseline selection and claim scope. The python-igraph interface [17] provides an accessible graph-library comparator. The DRMacIver/FAS tool [18], by contrast, is an external matrix-based ordering heuristic whose documented input model is pairwise-ordering data rather than sparse graph analysis. The evaluated executable exhibits run-to-run variation under internal initialization, so it is treated empirically as stochastic here. This makes it both a useful cross-paradigm comparator on sparse instances and an especially appropriate comparator on dense LOLIB instances, where the matrix-based formulation aligns naturally with the benchmark structure.

The dense linear-ordering literature provides the right background for the LOLIB transfer test. LOLIB was established as a benchmark family for complete weighted ordering problems rather than sparse general digraphs [19, 20]. Tournament and dense-ordering heuristics therefore form a separate comparison class, including weighted local-search methods [21] and more recent dense-ordering work such as [22]. The present paper uses LOLIB to mark a scope boundary rather than to claim superiority in that dense regime.

2.3 Minimal/stable FAS and SCC-based heuristics

SCC structure is a recurring theme in practical feedback arc heuristics because cycles are localized there. Decomposition along SCC boundaries is therefore a natural way to avoid spending effort on regions that are already acyclic. The present work uses that insight twice: in the attributed seed constructions and again in the IPSNS refinement layer, where only SCCs with positive incumbent backward contribution are eligible for repair.

The second seed in the framework comes from the minimal/stable feedback arc set line of Cavallaro and collaborators. Cavallaro and Cutello [7] study weighted minimal-and-stable feedback arc sets, extending earlier minimal/stable constructions [23]. That lineage matters here because it provides a complementary constructive bias to local-ratio seeding and because it distinguishes *minimal* from *minimum-weight*: an inclusion-minimal feedback arc set is one from which no arc can be restored while preserving acyclicity, but it need not minimize total weight.

This distinction is easy to blur in heuristic writing, so the paper keeps it explicit. LR-TA and the WMSF-style seed both use acyclicity-preserving add-back, and IPSNS inherits the same add-back logic inside SCC-restricted repair. What matters scientifically is not a claim of exact solution, but that these routines provide feasible and complementary seeds for later incumbent-protected refinement.

2.4 Relationship to preliminary author work and present contribution

The public preprint of Vahidi and Koutis [6] introduced the ranking-oriented framing of MWFAS together with an earlier local-ratio-based pipeline built around cycle peeling, feasible reinsertion, and topological extraction. The present manuscript does not claim that foundation as new. The LR-TA seed descends from that earlier pipeline and is attributed accordingly. The WMSF-style seed is likewise not presented as a new contribution of this paper; it is an adapted implementation of the weighted minimal/stable line of Cavallaro and Cutello [7].

What is new here is the IPSNS-centered framework and the accompanying computational study: dual-seed initialization, supporting feasibility and monotonicity analysis, exact and MIP validation, sparse benchmark comparisons, repeated-run analysis against the evaluated DRMacIver/FAS executable, holdout-supported defaults, and a dense LOLIB scope study. The contribution is therefore a sparse-digraph algorithm-engineering paper centered on incumbent-protected SCC-local refinement, with inherited components attributed rather than rebranded.

3 Problem definition

Let $G = (V, A, w)$ be a directed graph with vertex set V , arc set $A \subseteq V \times V$, and nonnegative arc weights $w : A \rightarrow \mathbb{R}_{\geq 0}$. A feedback arc set is a set $F \subseteq A$ such that removing F makes the graph acyclic. The minimum weighted feedback arc set problem asks for a minimum-weight feedback arc set, i.e., a set F minimizing $\sum_{a \in F} w(a)$.

An ordering π assigns each vertex a distinct position. An arc (u, v) is backward under π if u appears after v . The ordering-induced backward weight is

$$\text{bw}(\pi) = \sum_{(u,v) \in A: \pi(u) > \pi(v)} w(u, v). \quad (1)$$

For any ordering π , the set of backward arcs is a feedback arc set: all remaining forward arcs point from earlier to later positions and therefore form an acyclic graph. Conversely, every acyclic subgraph admits a topological order. The weighted feedback arc set problem can therefore be treated as the problem of finding an ordering with minimum backward weight [1, 2].

Many constructive heuristics in this study first build an acyclic *active* subgraph $H = (V, A \setminus F)$ by deleting a set $F \subseteq A$, and then extract a linear extension π of H . For such a pair (F, π) , define

$$B_\pi = \{(u, v) \in A : \pi(v) < \pi(u)\}. \quad (2)$$

Every active arc is forward under π , so $B_\pi \subseteq F$. For nonnegative weights,

$$\text{bw}(\pi) = w(B_\pi) \leq w(F). \quad (3)$$

Equality need not hold: topological extraction is generally *not* objective-neutral, because different linear extensions of the same active DAG can yield different values of $\text{bw}(\pi)$. The implementation therefore reports the ordering objective recomputed from the returned ranking, not the removed-set weight $w(F)$ alone. Online Resource 1 records the detailed extraction discussion and calibration checks.

Dense linear ordering problem benchmarks are often stated in terms of a complete weight matrix C , where an ordering $\pi = (\pi_1, \dots, \pi_n)$ is evaluated by the forward weight. In that notation,

$$\text{fw}_C(\pi) = \sum_{1 \leq i < j \leq n} C_{\pi_i, \pi_j}, \quad \text{bw}_C(\pi) = \sum_{1 \leq i < j \leq n} C_{\pi_j, \pi_i} = W_C - \text{fw}_C(\pi), \quad (4)$$

where $W_C = \sum_{i \neq j} C_{ij}$ is the total off-diagonal weight. Thus maximizing forward weight on a complete dense linear-ordering instance is equivalent to minimizing backward weight after converting the matrix to a complete weighted digraph. LOLIB results are reported as a transfer test. Those instances are dense complete ordering problems rather than sparse directed graph benchmarks.

4 Proposed methodology

The framework has three components: two attributed constructive seeds and one refinement layer. IPSNS is the central method. LR-TA and the WMSF-style routine are included because they provide complementary feasible starting points and because the IPSNS incumbent-protection guarantee is interpreted relative to the better of those two seeds. Online Resource 1 provides a compact framework schematic.

Algorithm 1 LR-TA seed construction

Require: Weighted digraph $G = (V, A, w)$ with $w \geq 0$
Ensure: Ordering π_{LR} , removed-edge set F_{LR}

```
1:  $F_{LR} \leftarrow \emptyset$ 
2: while the active graph contains a directed cycle  $C$  do
3:    $\varepsilon \leftarrow \min_{e \in C} W[e]$ 
4:   Subtract  $\varepsilon$  on  $C$ ; deactivate every newly tight active arc and add it to  $F_{LR}$ 
5: end while
6: Compute deterministic topological order  $\pi$  of the active graph
7: for all  $e = (u, v) \in F_{LR}$  in nonincreasing order of  $W_0[e]$  do
8:   if  $\text{rank}_\pi(u) < \text{rank}_\pi(v)$  or  $v \not\rightsquigarrow u$  in the current active graph then
9:     reactivate  $e$ ; remove  $e$  from  $F_{LR}$ 
10:    recompute  $\pi$  if the accepted arc is backward under the previous order
11:   end if
12: end for
13: return  $\pi_{LR}, F_{LR}$ 
```

All methods operate on the aggregated weighted digraph of Section 3 and optimize backward weight.

4.1 Local-ratio topological add-back

LR-TA is an engineered seed derived from the directed local-ratio lineage of [5] and from earlier author work [6]. It is not presented as a new approximation result. Its role in the present paper is to provide a strong sparse-graph seed whose output can later be improved, or preserved, by IPSNS.

Phase I repeatedly finds any directed cycle in the active graph and subtracts the minimum residual weight on that cycle from every arc in the cycle. If C is the detected cycle, the reduction amount is

$$\varepsilon(C) = \min_{e \in C} w(e). \quad (5)$$

At least one arc becomes tight and is removed from the active graph. Phase I ends when the active graph is acyclic.

Phase II performs one-pass heavy-first add-back. Removed arcs are processed in nonincreasing order of original weight. A removed arc is restored immediately if the current topological order already places it forward; otherwise it is restored only when a reachability test shows that reinsertion preserves acyclicity. Backward restorations trigger topological recomputation before later tests. The extracted topological order is deterministic under the chosen implementation rule, but it remains only one linear extension of the final DAG.

Algorithm 1 summarizes LR-TA.

The important point for the rest of the paper is that LR-TA delivers a feasible ordering together with a recorded removed set and a strong heavy-first repair stage. Extended implementation commentary, tolerance details, and proof expansions are deferred to Online Resource 1.

4.2 WMSF-style seed construction

The second seed is an adapted implementation of the weighted minimal/stable feedback arc set line of Cavallaro and Cutello [7]. It is carried as a complementary seed,

not as a separate novel claim. The routine decomposes the graph into SCCs, processes each nontrivial SCC independently, removes arcs according to either an $L1$ or $L2$ weight-based ordering, then applies heavy-first add-back, stabilization, and a second add-back pass. When the full graph is one nontrivial SCC, both $L1$ and $L2$ are tried and the better ordering is retained.

This seed provides a different bias from LR-TA: LR-TA is driven by cycle reduction in an active sparse graph, whereas the WMSF-style seed is driven by ordered removals and subsequent minimization/stabilization. Taking the better of the two before refinement therefore gives IPSNS a stronger incumbent than either seed alone. Step-by-step propagation details and source-to-code mapping appear in Online Resource 1.

4.3 Incumbent-protected SCC-neighborhood search

IPSNS is the primary contribution of the paper. Standard ingredients such as SCC decomposition, rollback, add-back, or destroy-and-repair are not claimed as new in isolation. What is new is their integration for sparse weighted MWFAS: fixed original-graph SCC neighborhoods, contribution-based SCC scoring, weighted top- K selection, two-sided perturbation, SCC-restricted local-ratio repair, SCC-restricted heavy-first add-back, and strict incumbent protection.

IPSNS first computes the two seeds and chooses the better ordering as the initial incumbent. At each iteration, it scores each nontrivial original-graph SCC by the backward weight currently contributed by internal arcs under the incumbent order. Only SCCs with positive contribution are eligible. One SCC is then sampled from the top- K eligible SCCs using score-proportional random choice.

Within the selected SCC, IPSNS executes a destroy-and-repair move. The destroy stage temporarily reactivates a heavy fraction of currently removed internal arcs and removes a light fraction of currently active internal arcs. The repair stage reruns local-ratio cycle reduction and one-pass heavy-first add-back only on the chosen SCC neighborhood, with original weights restored on that neighborhood. All edits outside the neighborhood remain fixed.

The candidate is accepted only if it induces a valid global ordering and strictly decreases backward weight. Otherwise the pre-move state is restored exactly. IPSNS therefore never returns a worse solution than the better seed from which it started. The method is stochastic because SCC selection and perturbation choices depend on the configured random seed, but it is reproducible for fixed input, parameter settings, and seed.

Algorithm 2 summarizes the driver.

The parameterization is interpreted conservatively in this paper. Reported defaults are holdout-supported engineering choices, not claims of universal optimality. Full parameter grids, additional implementation commentary, and supplementary controls are deferred to Online Resource 1.

4.4 Supporting correctness properties and complexity

The following properties formalize feasibility and monotonicity of the implemented framework; they are supporting guarantees rather than new approximation results.

Algorithm 2 IPSNS with incumbent protection

Require: Weighted digraph $G = (V, A, w)$, iteration budget T
Ensure: Ordering π^* with objective no worse than the best internal seed

```

1: Build WMSF-style seed  $\pi_W$  and LR-TA seed  $\pi_{LR}$ 
2:  $\pi^* \leftarrow \arg \min \{ \text{bw}(\pi_W), \text{bw}(\pi_{LR}) \}$ 
3: for  $t = 1, \dots, T$  do
4:   Score each nontrivial SCC by its current backward contribution
5:   if all SCC scores are zero then
6:     break
7:   end if
8:   Sample one SCC from the top- $K$  positive-score SCCs using weighted random choice
9:   Save the current SCC state; reactivate heavy removed edges; remove light active edges
10:  Apply SCC-restricted local-ratio repair and heavy-first add-back
11:  if the candidate SCC state is invalid then
12:    restore saved SCC state; continue
13:  end if
14:  Recompute the global ordering and candidate objective
15:  if the candidate objective is smaller than  $\text{bw}(\pi^*)$  then
16:    accept the candidate and update  $\pi^*$ 
17:  else
18:    restore the previous incumbent exactly
19:  end if
20: end for
21: return  $\pi^*$ 

```

Detailed proofs are deferred to Online Resource 1 (§S4). Throughout, $n = |V|$, $m = |A|$, T is the IPSNS iteration budget, and for a selected SCC S we write $n_S = |V(S)|$ and $m_S = |A(S)|$.

Proposition 1 (LR-TA feasibility and progress) *Assume nonnegative original weights and the active-graph representation used in the implementation. Each Phase-I cycle reduction subtracts $\varepsilon(C)$ from every arc in a detected directed cycle C , so at least one active arc becomes tight and is deactivated. Phase I therefore terminates after finitely many iterations and leaves an acyclic active graph. Phase II restores a removed arc only when the current topological order already places it forward or when a reachability test certifies that restoration preserves acyclicity. The final active graph is acyclic and induces a feasible ordering for the objective of Section 3.*

Proposition 2 (One-pass add-back minimality) *Let $G' = (V, A')$ be the current active acyclic graph with topological order π . After every candidate removed arc has been processed once and reinserted whenever insertion preserves acyclicity, the remaining removed set F is inclusion-minimal in the feedback-arc-set sense:*

$$\forall e \in F, \quad G' + e \text{ is cyclic.} \quad (6)$$

Equivalently, for every $e \in F$, restoring e to A' creates a directed cycle, or $A' \cup \{e\}$ is cyclic.

If inserting $e = (u, v)$ is rejected, a directed path $v \rightsquigarrow u$ already exists in the current active graph. Later accepted insertions only add arcs and cannot destroy that path, so e cannot become feasible later in the same pass. Heavy-first ordering therefore affects which inclusion-minimal feedback arc set is obtained, not whether inclusion-minimality holds. Inclusion-minimality does not imply minimum weight, and this statement applies to one-pass acyclicity-preserving add-back rather than to WMSF stabilization swaps.

Proposition 3 (IPSNS incumbent non-worsening) *Let $\pi^{(0)}$ be the better of the LR-TA and WMSF-style seeds, and let $\pi^{(t)}$ denote the incumbent after iteration t of IPSNS. Then*

every accepted candidate is feasible, every infeasible or non-improving candidate is rolled back exactly, and

$$\text{bw}(\pi^{(t+1)}) \leq \text{bw}(\pi^{(t)}) \leq \text{bw}(\pi^{(0)}) \quad \text{for all } t \geq 0. \quad (7)$$

The outer loop executes at most T iterations and may terminate earlier when every eligible SCC has zero backward contribution.

The proof logic is straightforward. LR-TA progresses because each detected cycle reduction tightens at least one active arc. One-pass add-back is valid because rejected arcs remain blocked by an already existing reverse path. IPSNS is monotone because every failed or non-improving move is rolled back exactly before the next iteration.

Complexity summary.

The implementation uses sparse edge-indexed storage, so space usage is $O(n + m)$. LR-TA Phase I performs repeated cycle detection in the active graph and Phase II performs one-pass add-back, giving the conservative bound

$$O(m(n + m)) + O(r \log r + r(n + m)), \quad (8)$$

where r is the number of removed arcs examined in add-back. The WMSF-style seed first decomposes the graph into SCCs and then performs ordered removals, add-back, and stabilization inside each nontrivial SCC, yielding the conservative aggregate bound

$$O\left(n + m + \sum_S (m_S \log m_S + r_S(n_S + m_S))\right), \quad (9)$$

with r_S the number of add-back candidates processed in SCC S . One IPSNS iteration scores SCCs in $O(m)$, selects one SCC, performs SCC-restricted destroy-and-repair, and then recomputes the global objective, so a conservative per-iteration bound is

$$O(m + s \log s + c_S(n_S + m_S) + r_S \log r_S + r_S(n_S + m_S)), \quad (10)$$

where s is the number of eligible SCCs and c_S is the number of restricted cycle-reduction rounds. These are conservative implementation-level bounds; practical runtime depends strongly on SCC structure, density, and the selected iteration budget.

5 Experimental design

The experiments are designed to support a scoped claim about sparse nonnegative weighted digraphs, not a universal claim across all feedback arc set regimes. The sparse graph-benchmark collection provides the primary claim-bearing setting. Exact and MIP validation provide certified reference points on subsets where certification is feasible. EXP10 evaluates repeated-run behavior against an external executable whose initialization varies across runs. LOLIB serves as a transfer test that marks the dense complete-ordering boundary. Tables 1 and 2 summarize the comparison surface.

Table 1: Baseline provenance and comparison scope.

Method	Role	Origin	External in-repo	/ Problem model	Weighted	Implementation behavior	Evaluation subset
LR-TA	internal seed	[5, 6]	in-repo	sparse FAS / ordering	yes	deterministic fixed input tie-breaking deterministic fixed input	for and sparse, exact, hold-out
WMSF-style seed	internal seed	adapted from [7, 23]	in-repo adaptation	sparse FAS	yes	deterministic fixed input	for sparse, exact, LOLIB
IPSNS	proposed method	this study	in-repo	sparse FAS refinement	yes	stochastic reproducible for fixed configuration and seed	but for sparse, exact, hold-out, LOLIB
Borda / net score	calibration	[16]	in-repo	ordering score rule	yes	deterministic	sparse, LOLIB
Weighted Eades adaptation	adaptation	adapted from [12, 13]	in-repo adaptation	sparse rule ordering heuristic	yes	deterministic	sparse, LOLIB
Random multistart	calibration	standard multistart	in-repo	ordering heuristic	yes	stochastic	sparse, LOLIB
igraph Eades	external baseline	[17]	external maintained implementation	graph-library FAS heuristic	yes	deterministic evaluated	as sparse, LOLIB
DRMacIver/FAS	external baseline	[18]	external executable	matrix ordering	yes	evaluated executable run-to-run variation under internal initialization	sparse, EXP10, LOLIB, EXP9
Exact DP	exact validator	repository implementation	in-repo	sparse $n \leq 20$	yes	deterministic	exact validation
HiGHS MIP	exact validator	[24] via <code>scipy.optimize.milp</code>	external maintained implementation	sparse-digraph MIP	yes	deterministic under fixed settings	medium MIP

Table 2: Comparison accounting for the main experiments.

Experiment	Purpose	Instances attempted	Common completed subset	/ Runs per method	Timeout	Primary metric	Statistical unit
Primary sparse (EXP4)	main sparse comparison	97 std. nonneg.	97 internal; 93 paired with DR	1	wrapper; 4 DR incomplete $n \leq 20$	BW	instance
Exact DP (EXP3)	small-instance certification	57	57 certified optima	1	120 s	gap to exact optimum	instance
MIP validation (EXP8)	medium-instance certification	15	7 proven optimal, time-limited	8	120 s	gap when certified	instance
Holdout check	default selection	43 total	25 holdout instances	5	none	final by configuration	instance/config aggregate
Repeated-run (EXP10)	typical repeated-run behavior	93	93 common sparse instances	20	EXP4 wrappers	median BW	paired instance difference
Dense transfer (EXP5)	complete-ordering boundary	50 LOLIB	50	1	standard wrappers	BW	instance

5.1 Benchmarks and scope

The primary benchmark family comes from the public **graph-benchmarks** collection [25]. After path-level deduplication, it yields 105 weighted sparse DIMACS digraph instances. The main benchmark tables restrict attention to the 97 standard instances whose aggregated arc weights are nonnegative. Eight instances are excluded from the standard set because they contain negative-weight arcs: **gerez**, **howard-max**, **k3_3**, **ku**,

`peterson`, `peterson1`, `peterson2`, and `stg0`. This is a methodological scope restriction rather than a favorable filter: the local-ratio construction and the interpretation of incumbent protection used here are stated for nonnegative weights.

Three benchmark subsets play supporting roles. First, the exact-validation subset consists of the 57 standard nonnegative instances with $n \leq 20$, on which bitmask dynamic programming can certify the optimum. Second, a medium-scale subset of 15 instances supports the time-capped MIP validation. Third, a parameter study splits 43 sparse instances into 18 tuning instances and 25 holdout instances; its purpose is only to check whether the chosen IPSNS defaults are defensible engineering choices.

The dense transfer benchmark is a 50-instance subset of LOLIB 2010 [19, 20], comprising 25 SGB, 10 IO, and 15 RandA1 instances. These are complete weighted ordering instances rather than sparse general digraphs. They are included to delimit regime dependence: sparse-benchmark success should not be interpreted automatically as dense-ordering dominance.

5.2 Methods compared

IPSNS is the primary method under study. It starts from the better of two internal seeds and applies incumbent-protected SCC-local refinement. The two seeds are LR-TA and a WMSF-style adaptation of the weighted minimal/stable line [7, 23]. They are retained in the tables because they clarify what the refinement adds and because the IPSNS non-worsening statement is relative to those seeds.

External and calibration baselines are chosen to cover accessible comparison classes rather than to exhaust the ranking literature. DRMacIver/FAS [18] is an external matrix-based ordering heuristic and the main external comparator in the sparse study. The evaluated executable exhibits run-to-run variation under internal initialization, which motivates EXP10. The python-igraph interface [17] supplies a graph-library Eades-style baseline. Weighted Eades, Borda/net-score, and random multistart are included as transparent local calibrators. Exact DP is used only on the 57-instance validation subset, and HiGHS MIP is used only for the medium-instance certification study.

5.3 Evaluation metrics and accounting rules

Backward weight (BW) is the primary metric throughout. Every method is evaluated under the same BW objective on the original aggregated arc weights, regardless of whether the method internally operates on an ordering, a removed arc set, or a matrix representation.

The sparse and dense comparison tables also report *Best*, meaning the number of instances on which a method attains the minimum observed BW among the evaluated methods. Ties are credited to all tied methods. Relative excess over IPSNS is the mean percentage reduction achieved by IPSNS under the baseline-normalized convention $(X - \text{IPSNS})/X \times 100$, where X is the comparator BW on the completed paired instances. Exact-validation tables instead report optimal-match counts and a mean gap normalized by total arc weight, matching the generator; the lone nonzero instance

is discussed explicitly in the results text. Runtime is reported only as contextual wall-clock information and is not normalized across different external implementations.

Common-subset accounting is enforced wherever completion differs across methods. In the primary sparse comparison, DRMacIver/FAS completes 93 of the 97 standard instances; the remaining four incompletions remain visible rather than being hidden inside aggregate tables. The EXP10 repeated-run study therefore uses the 93-instance common subset completed by both IPSNS and DRMacIver/FAS. The MIP study reports certified gaps only on the 7 instances proved optimal within the time limit and treats the remaining 8 time-limited cases as incomplete reference evidence rather than heuristic failures.

5.4 Statistical treatment

Paired comparisons use the instance as the statistical unit. For EXP4 and EXP10, the paired outcome is the per-instance difference in BW between two methods on the relevant common subset. Archived summaries report two-sided Wilcoxon signed-rank and sign tests; the implemented Wilcoxon call uses SciPy’s zero-difference removal convention (`zero_method="wilcox"`), and the sign test excludes ties from the win/loss count. For EXP10 this leaves 38 nonzero paired median differences and 55 zero differences. Bootstrap confidence intervals are paired, use seed 42, and use 10,000 resamples. Relative excess in EXP10 is computed from per-instance medians using the same convention as EXP4:

$$(\text{DR} - \text{IPSNS})/\text{DR} \times 100. \quad (11)$$

The archived summaries document the two-sided tests and bootstrap configuration directly; they do not materialize the Wilcoxon implementation mode, so the manuscript does not over-specify it.

5.5 Repeated-run and holdout protocols

EXP10 is a confirmatory repeated-run comparison on the 93-instance common sparse subset. Each instance receives 20 IPSNS runs with explicit integer seeds and 20 DRMacIver/FAS repetitions through the same wrapper and BW recomputation pipeline used in the main sparse comparison. The primary analysis compares per-instance median BW values with tie tolerance 10^{-9} . The protocol and analysis plan were fixed before examining aggregate outcomes. Reported summaries include wins, ties, losses, paired mean and median differences, mean relative excess, test results, effect size, and bootstrap intervals. Full per-instance summaries appear in Online Resource 1.

The holdout study is intentionally modest. It is not a claim-bearing benchmark comparison but a conservative check on engineering defaults. Across 18 tuning and 25 holdout sparse instances, six parameter configurations were each run with five seeds per instance, for 1290 archived runs in total. The holdout aggregate did not separate the tested defaults on median final BW, so the chosen IPSNS settings are interpreted as holdout-supported defaults rather than tuned optima.

Table 3: Primary sparse comparison on the 97 standard nonnegative instances. BW denotes backward weight; lower values are better. Best counts every method attaining the minimum observed BW on an instance; ties are credited to all tied methods. Relative excess is the mean percentage reduction achieved by IPSNS under the baseline-normalized convention $(X - \text{IPSNS})/X \times 100$, where X is the comparator BW on completed instances.

Method	Complete	Mean BW	Mean RT (s)	Best	Relative excess
IPSNS (ours)	97/97	37,698	21.92	96	0.00%
LR-TA (ours)	97/97	38,327	0.08	80	0.71%
WMSF seed	97/97	40,005	1.31	61	2.06%
DRMacIver/FAS	93/97	53,173	4.00	56	21.61%
igraph Eades	97/97	95,920	0.01	40	30.54%
Weighted Eades	97/97	99,689	0.11	42	30.48%
Borda net score	97/97	512,277	0.00	27	55.55%
Random multistart	97/97	1,075,258	0.03	42	49.76%

DRMacIver/FAS is the main external comparator on this benchmark, but it completes 93 of 97 instances because two DAG cases return empty-tournament errors and two large sparse instances time out in the wrapper used here. Its mean relative excess of 21.61% is a comparator-normalized reduction relative to IPSNS, not an IPSNS-normalized percent-worse score.

5.6 Implementation environment and reproducibility scope

All reported numbers are taken from archived experiment outputs and regenerated manuscript assets. The runs used a Linux workstation with an Intel Core i7-12700K CPU, 62 GiB RAM, Ubuntu 24.04, and Python 3.12. External tools were invoked through documented interfaces or wrappers, and failures or incompletions were retained explicitly in the summaries. Online Resource 1 supplies the deferred protocols, configurations, processed outputs, proofs, and regeneration scripts. Full reruns additionally require the documented external data, binaries, and computational resources.

6 Computational results

The results are organized around the paper’s claim hierarchy. The primary question is how IPSNS performs on the standard sparse nonnegative benchmark. The supporting questions are whether that performance survives exact validation, how repeated-run behavior compares with the strongest external executable, what the ablation evidence says about the integrated design, and where the sparse-graph advantage stops transferring.

6.1 Primary sparse comparison

Table 3 reports the main comparison on the 97 standard nonnegative sparse instances.

IPSNS attains the minimum observed BW among the evaluated methods on 96 of the 97 instances. LR-TA and the WMSF-style seed are the strongest internal comparators, but both trail IPSNS in mean BW and best-count frequency. Among

Table 4: Exact validation on the 57 standard non-negative instances with $n \leq 20$. Mean gap follows the generator’s total-weight normalization, while the single miss on `r20_60` is discussed in the text using the exact-optimum denominator.

Method	Optimal	Optimality rate	Mean gap
IPSNS (ours)	56/57	98.2%	0.0006%
LR-TA (ours)	55/57	96.5%	0.0590%
WMSF seed	51/57	89.5%	0.0961%

This table supports empirical near-optimality on small instances only; it does not imply a general approximation guarantee. The only IPSNS near-miss is instance `r20_60`, where the BW gap is 3 and the optimum-normalized gap is 0.178% even though the table’s mean-gap convention averages gap/total weight.

the external methods, DRMacIver/FAS is the strongest comparator on this benchmark, and the generator reports a mean relative reduction of 21.61% for IPSNS under the baseline-normalized convention. The incompleteness matter and are kept explicit: DRMacIver/FAS completes 93 of the 97 instances because two DAG inputs are rejected by the wrapper and two large sparse instances time out.

On the full 105-instance sparse benchmark, IPSNS records zero incumbent violations relative to the better seed. The refinement layer therefore improves selectively rather than trading improvements for regressions. LR-TA remains the clear low-latency option, with mean runtime 0.08 seconds per instance and mean BW close to IPSNS, but IPSNS is the method that secures the 96-of-97 sparse result.

Single-run paired outcomes against DRMacIver/FAS on the 93 common standard instances are 37 wins, 55 ties, and 1 loss for IPSNS. Because those counts are tie-heavy and the external executable varies across repetitions, the more informative claim comes from the repeated-run analysis in Section 6.3. Full sparse paired-test tables for all baselines are moved to Online Resource 1.

6.2 Exact validation, MIP checks, and holdout confirmation

Table 4 gives the exact small-instance validation results.

IPSNS matches the exact dynamic-programming optimum on 56 of the 57 standard nonnegative instances with $n \leq 20$. LR-TA matches on 55 and the WMSF-style seed on 51. The exact-validation table uses the committed generator’s total-weight normalization, so the reported mean gap is 0.0006% even though the sole miss on `r20_60` has a 3-unit BW gap, corresponding to a 0.178% optimum-normalized gap. The correct interpretation is empirical near-optimality on the validated subset, not a new optimality guarantee for larger graphs.

The time-capped MIP study extends that picture slightly beyond the DP regime. On 15 selected medium instances, HiGHS certifies optimality on 7 and times out on 8. IPSNS matches the certified optimum on 6 of the 7 solved cases; again, the sole mismatch is `r20_60`, with IPSNS BW 1688 versus certified MIP optimum 1685 and

Table 5: Repeated-run comparison (EXP10) on the 93-instance common sparse subset completed by both IPSNS and DRMacIver/FAS. Each method was run 20 times per instance; outcomes use per-instance median backward weight (BW) with tie tolerance 10^{-9} . Relative excess uses $(\text{DR} - \text{IPSNS})/\text{DR} \times 100$ on paired medians.

Method	Reps	Median W/T/L	Distinct BW	Stat. test
IPSNS	20	38/55/0	1 value on 93/93 inst.	Wilcoxon $p < 0.001$
DRMacIver/FAS	20	—	>1 value on 40/93 inst.	Sign test $p < 0.001$

W/T/L counts IPSNS wins, ties, and losses versus DRMacIver/FAS on per-instance medians. Mean paired relative excess of DRMacIver/FAS over IPSNS is 21.6% (unchanged from the EXP4 single-run figure). Cohen’s $d_z = -0.31$ for paired median differences; 95% bootstrap CI for mean paired difference in BW units is $[-41,787, -10,718]$. The median paired relative excess is 0.0% because 55 of 93 instances are exact ties under the median comparison. IPSNS showed zero cross-seed objective variation under the fixed configuration; this documents empirical stability, not mathematical determinism.

an optimum-normalized gap of 0.178%. The remaining 8 cases are reported only as incomplete certification attempts. Detailed per-instance MIP results are deferred to Online Resource 1.

The holdout parameter study is intentionally conservative. Across 18 tuning and 25 holdout instances, six IPSNS configurations produced the same holdout aggregate median final BW under the archived five-seed protocol. The paper therefore keeps the default configuration as a reasonable engineering choice rather than claiming that the selected parameters are uniquely optimal.

6.3 Repeated-run comparison with DRMacIver/FAS

Table 5 summarizes EXP10 on the 93-instance common sparse subset.

The per-instance median comparison yields 38 wins, 55 ties, and no losses for IPSNS. Thus, the typical common instance is a tie, but the non-tied outcomes are strongly asymmetric in favor of IPSNS, and several large improvements produce a substantial mean reduction relative to DRMacIver/FAS. The median paired relative excess is 0.0% because ties dominate, while the mean paired relative excess of DRMacIver/FAS over IPSNS remains 21.6% under the baseline-normalized convention.

The archived summaries report two-sided Wilcoxon signed-rank and sign tests on paired per-instance median differences, both with $p < 0.001$. For the sign test, ties are excluded from the win/loss count, leaving 38 nonzero paired median differences, all in favor of IPSNS. IPSNS shows one distinct BW value on all 93 instances across 20 seeds under the fixed configuration. That is evidence of empirical stability for this configuration, not proof of mathematical determinism. DRMacIver/FAS, by contrast, shows run-to-run objective variation on 40 of the 93 instances, which is precisely why a single execution is not always representative for that executable.

6.4 Ablation evidence

The ablation study on the 10-instance representative sparse subset supports two points. First, topological add-back is the dominant seed-side design choice: removing it degrades mean BW by 5.94% relative to LR-TA. Second, IPSNS provides a further

Table 6: Dense LOLIB transfer test on 50 complete weighted ordering instances. Lower BW is better. Best counts every method attaining the minimum observed BW on an instance; ties are credited to all tied methods. Family-level detail is deferred to Online Resource 1.

Method	Mean BW	Best	Mean RT (s)
DRMacIver/FAS	571,687	45/50	1.01
IPSNS (ours)	582,354	5/50	37.92
LR-TA (ours)	582,948	2/50	0.22
WMSF seed	585,926	1/50	0.19
igraph Eades	659,570	0/50	0.01
Weighted Eades	675,235	0/50	0.01
Random multistart	883,664	0/50	0.02
Borda net score	1,066,045	0/50	0.00

DRMacIver/FAS is tournament-native and is the best overall dense LOLIB method in this study (45/50 instances, 3.88% lower mean BW than IPSNS). EXP5 is therefore a transfer test and scope boundary.

reduction of about 0.76% after seeding, and the 50- and 400-iteration variants have equal mean BW on this subset. That second gain is smaller, but it is achieved after a strong seed and under strict non-worsening. The full ablation table and secondary controls are moved to Online Resource 1.

6.5 Dense LOLIB boundary

Table 6 reports the dense transfer test.

The dense result draws the regime boundary cleanly. DRMacIver/FAS attains the minimum observed BW on 45 of the 50 LOLIB instances, whereas IPSNS does so on 5. The mean BW gap is 3.88% in favor of DRMacIver/FAS. IPSNS remains second overall, but the dense ordering regime favors the matrix-based comparator.

This outcome is consistent with the problem structure rather than contradictory to the sparse story. IPSNS is built around sparse weighted-digraph seeding and SCC-local repair. LOLIB consists of complete weighted ordering instances, so the cyclic structure is much less localized and matrix-based pairwise-ordering methods are better aligned with the input model. The LOLIB test therefore serves as an explicit scope boundary for the main sparse-graph claim.

7 Discussion

7.1 Why SCC-local refinement helps on sparse graphs

The sparse-benchmark results fit the intended mechanism of IPSNS. LR-TA and the WMSF-style seed already produce strong feasible orderings, but they do so through different biases: LR-TA emphasizes cycle reduction and heavy-first repair, while the WMSF-style seed emphasizes ordered removals and stabilization. IPSNS inherits the better of those two constructions and then spends its search budget only where backward weight still remains concentrated.

That focus matters on sparse graphs because the difficult part of the objective is often localized. Large acyclic or weakly cyclic regions do not justify broad perturbation. The ablation evidence is consistent with that interpretation: most of the gain comes from engineering a strong seed, and IPSNS then improves the harder residue without ever sacrificing the better seed. The gains can be numerically modest because the starting point is already strong, but consistency matters here more than large changes on every instance.

The repeated-run comparison sharpens this interpretation. The EXP10 median outcomes are tie-heavy, so the typical common instance does not show a dramatic change between IPSNS and DRMacIver/FAS. But the non-tied outcomes are one-sided in favor of IPSNS, and the aggregate mean reduction remains large. That is the profile one would expect from a protected refinement layer: many instances are already effectively solved by the seed or are insensitive to local refinement, while a smaller subset carries the material gains.

7.2 Sparse versus dense regimes

The LOLIB transfer test is not a contradiction to the sparse result; it identifies the boundary of the method’s intended operating regime. Dense complete ordering instances collapse much of the distinction between cyclic core and acyclic background. When every ordered pair carries weight, a matrix-based pairwise-ordering heuristic is working with a model that is better aligned to the benchmark than a sparse-graph SCC-local heuristic.

This is why the paper keeps the dense result visible. IPSNS remains second overall on LOLIB, yet the overall dense result still belongs to DRMacIver/FAS. The sparse claim is therefore strong but bounded: IPSNS is a useful method for sparse nonnegative weighted digraphs among the evaluated methods, not a universal winner across ordering models.

7.3 Practical implications

The experiments suggest a simple practical division of labor. LR-TA is the right choice when runtime is tight and a strong deterministic sparse-graph seed is sufficient. The WMSF-style seed is useful because it broadens the initial solution set and occasionally provides the better incumbent from which refinement should start. IPSNS is preferable when solution quality on sparse nonnegative weighted digraphs matters more than seed-level latency and when exact rollback is desirable. Matrix-based methods such as DRMacIver/FAS are better choices on dense complete ordering inputs.

The study also suggests a methodological point for future heuristic design. Much of the value comes not from inventing entirely new primitive operations, but from integrating known ingredients under a disciplined acceptance rule.

7.4 Limitations

1. IPSNS is a heuristic framework with supporting feasibility and monotonicity properties; it does not come with a new approximation guarantee.

2. The main claim is restricted to nonnegative weighted digraphs; negative-weight instances are outside the setting in which the present local-ratio and incumbent-protection interpretation is used.
3. The external baseline ecosystem is fragmented: some relevant methods lack accessible weighted implementations in a comparable pipeline, so the paper’s strongest claims are explicitly relative to the evaluated methods and subsets.
4. Runtime comparisons depend on implementation choices, data representation, and external wrappers; the quality comparisons are cleaner than any claim of machine-independent speed ranking.
5. Dense complete ordering remains a weaker regime for IPSNS than for matrix-based pairwise-ordering heuristics, as the LOLIB results show.
6. Zero observed IPSNS variation across 20 seeds in EXP10 documents empirical stability for the tested configuration, not mathematical determinism.
7. Full benchmark reruns require the documented external data, binaries, and computational resources distributed across the companion resource package.

Within those limits, the main conclusion remains intact: incumbent-protected SCC-local refinement is a useful algorithm-engineering strategy for sparse nonnegative weighted digraphs. It builds on attributed seed methods, improves them selectively, and reports clearly where that advantage does not transfer.

8 Conclusions

This paper studied MWFAS in the regime of sparse nonnegative weighted digraphs. The central contribution is IPSNS, an incumbent-protected SCC-local destroy-and-repair heuristic that starts from the better of two attributed seeds, concentrates refinement on SCCs that still contribute backward weight, and accepts a candidate only when the global objective strictly improves. The seeds themselves are supporting components: a local-ratio topological add-back construction rooted in prior local-ratio work and earlier author work, and a WMSF-style seed adapted from Cavallaro and Cutello’s weighted minimal/stable line.

The main empirical result is deliberately scoped. On the primary sparse benchmark, IPSNS attains the minimum observed backward weight among the evaluated methods on 96 of 97 standard nonnegative instances. On the exact-validation subset, it matches the dynamic-programming optimum on 56 of 57 instances, with the only near-miss again occurring on `r20_60`. On the 93-instance common sparse subset, repeated-run per-instance medians over 20 runs give 38 wins, 55 ties, and 0 losses versus the evaluated DRMacIver/FAS executable. These results support the interpretation of IPSNS as a consistent refinement layer rather than a one-off benchmark outlier.

The supporting studies also mark the method’s boundary. Heavy-first add-back is the dominant seed-side design choice, IPSNS adds a smaller but meaningful improvement after strong seeding, and the rollback rule ensures that this gain never comes at the cost of a worse final solution than the better seed. The dense LOLIB transfer test then shows where the sparse advantage stops transferring: matrix-based ordering methods remain preferable on complete weighted ordering instances. Future

work should test whether denser or hybrid seed constructions can extend the same incumbent-protected refinement discipline beyond the sparse setting.

Statements and Declarations

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests

The author declares that there are no known competing financial or non-financial interests that could have appeared to influence the work reported in this paper.

Author contributions

Soroush Vahidi: Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Related manuscripts and prior author work

This manuscript extends related author work on minimum weighted feedback arc set and ranking-as-ordering heuristics. Section 2.4 describes the relationship to the public preprint [6] and explains how the LR-TA and WMSF-style components are attributed. No substantially overlapping manuscript is currently under consideration elsewhere.

Data and code availability

Online Resource 1 accompanies the manuscript and contains the source code, parameter configurations, processed outputs, extended tables, deferred proofs, and scripts for regenerating the principal manuscript tables and figures. It also includes a test suite, small-instance cross-checks, and the processed exact and MIP validation materials used in the supporting studies. The public benchmark families used in the paper are cited in the manuscript, and Online Resource 1 documents the corresponding acquisition and preprocessing procedures.

Reproducing the full benchmark runs additionally requires the documented external data, third-party tools, and computational resources described in the supplementary material. The package is intended to make the reported evidence traceable without requiring every reader to rerun the most expensive computations.

Generative AI and AI-assisted technologies

During the preparation of this work, the author used AI-assisted tools, including ChatGPT, Codex, Claude, and Perplexity AI, to support literature exploration, organization of material, language editing, and coding assistance. The author reviewed

and edited all outputs, verified the relevant sources and experimental results, and takes full responsibility for the content of the submitted manuscript.

References

- [1] Karp, R.M.: Reducibility among combinatorial problems. In: Complexity of Computer Computations, pp. 85–103. Springer, Boston, MA (1972). https://doi.org/10.1007/978-1-4684-2001-2_9
- [2] Flood, M.M.: Exact and heuristic algorithms for the weighted feedback arc set problem: A special case of the skew-symmetric quadratic assignment problem. *Networks* **20**(1), 1–23 (1990) <https://doi.org/10.1002/net.3230200102>
- [3] Ailon, N., Charikar, M., Newman, A.: Aggregating inconsistent information: Ranking and clustering. *Journal of the ACM* **55**(5), 1–27 (2008) <https://doi.org/10.1145/1411509.1411513>
- [4] Baharev, A., Schichl, H., Neumaier, A., Achterberg, T.: An exact method for the minimum feedback arc set problem. *ACM Journal of Experimental Algorithmics* **26**, 1–28 (2021) <https://doi.org/10.1145/3446429>
- [5] Demetrescu, C., Finocchi, I.: Combinatorial algorithms for feedback problems in directed graphs. *Information Processing Letters* **86**(3), 129–136 (2003) [https://doi.org/10.1016/S0020-0190\(02\)00491-X](https://doi.org/10.1016/S0020-0190(02)00491-X)
- [6] Vahidi, S., Koutis, I.: Ranking from Pairwise Comparisons as Minimum Weighted Feedback Arc Set (2024). <https://doi.org/10.48550/arXiv.2412.16181> . <https://arxiv.org/abs/2412.16181>
- [7] Cavallaro, C., Cutello, V.: An efficient heuristic algorithm to compute minimal and stable weighted feedback arc sets. In: Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE), pp. 84–87 (2025). <https://doi.org/10.18293/SEKE2025-049>
- [8] Lempel, A., Cederbaum, I.: Minimum feedback arc and vertex sets of a directed graph. *IEEE Transactions on Circuit Theory* **13**(4), 399–403 (1966) <https://doi.org/10.1109/TCT.1966.1082620>
- [9] Bar-Yehuda, R., Geiger, D., Naor, J.S., Roth, R.M.: Approximation algorithms for the feedback vertex set problem with applications to constraint satisfaction and bayesian inference. *SIAM Journal on Computing* **27**(4), 942–959 (1998) <https://doi.org/10.1137/S0097539796305109>
- [10] Even, G., Naor, J.S., Schieber, B., Sudan, M.: Approximating minimum feedback sets and multicuts in directed graphs. *Algorithmica* **20**(2), 151–174 (1998) <https://doi.org/10.1007/PL00009191>

- [11] Hecht, M., Gonciarz, K., Horvát, S.: Tight localizations of feedback sets. *ACM Journal of Experimental Algorithmics* **26**, 1–19 (2021) <https://doi.org/10.1145/3447652>
- [12] Eades, P., Lin, X., Smyth, W.F.: A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters* **47**(6), 319–323 (1993) [https://doi.org/10.1016/0020-0190\(93\)90079-O](https://doi.org/10.1016/0020-0190(93)90079-O)
- [13] Eades, P., Lin, X.: A heuristic for the feedback arc set problem. *Australasian Journal of Combinatorics* **12**, 15–25 (1995)
- [14] Brandenburg, F.J., Hanauer, K.: Sorting heuristics for the feedback arc set problem. In: *WALCOM: Algorithms and Computation. Lecture Notes in Computer Science*, vol. 7748, pp. 1–12. Springer, Berlin, Heidelberg (2013)
- [15] Simpson, M., Srinivasan, V., Thomo, A.: Efficient computation of feedback arc set at web-scale. *Proceedings of the VLDB Endowment* **10**(3), 133–144 (2016) <https://doi.org/10.14778/3021924.3021930>
- [16] Coppersmith, D., Fleischer, L.K., Rurda, A.: Ordering by weighted number of wins gives a good ranking for weighted tournaments. *ACM Transactions on Algorithms* **6**(3), 1–13 (2010) <https://doi.org/10.1145/1798596.1798608>
- [17] python-igraph developers: python-igraph documentation: `Graph.feedback_arc_set`. <https://python.igraph.org/en/stable/api/igraph.Graph.html>. Documentation for the weighted feedback arc set API; accessed 2026-06-06 (2026)
- [18] DRMacIver: Feedback-Arc-Set. <https://github.com/DRMacIver/Feedback-Arc-Set>. Software repository; experiment commit 16ff24a92fde886e58819180a9fe686e60991c5c; accessed 2026-06-06 (2026)
- [19] Marti, R., Reinelt, G., Duarte, A.: A benchmark library and a comparison of heuristic methods for the linear ordering problem. *Computational Optimization and Applications* **51**(3), 1297–1317 (2012) <https://doi.org/10.1007/s10589-010-9384-9>
- [20] Marti, R.: LOLIB: Linear Ordering Problem Library. <https://www.uv.es/~rmarti/paper/lop.html>. Official library page; accessed 2026-06-06 (2010)
- [21] Fomin, F.V., Lokshtanov, D., Raman, V., Saurabh, S.: Fast local search algorithm for weighted feedback arc set in tournaments. *Proceedings of the AAAI Conference on Artificial Intelligence* **24**(1), 65–70 (2010) <https://doi.org/10.1609/aaai.v24i1.7557>
- [22] Ostovari, M., Zarei, A.: A better LP rounding for feedback arc set on tournaments. *Theoretical Computer Science* **1015**, 114768 (2024) <https://doi.org/10.1016/j.tcs>

2024.114768

- [23] Cavallaro, C., Cutello, V., Pavone, M.: Efficient heuristics to compute minimal and stable feedback arc sets. *Journal of Combinatorial Optimization* **48**(30) (2024) <https://doi.org/10.1007/s10878-024-01209-8>
- [24] Huangfu, Q., Hall, J.: Parallelizing the dual revised simplex method. *Mathematical Programming Computation* **10**(1), 119–142 (2018) <https://doi.org/10.1007/s12532-017-0130-5>
- [25] Ali Dasdan: graph-benchmarks: Directed Graph Benchmarks in DIMACS Graph Format. <https://github.com/alidasdan/graph-benchmarks>. Dataset repository; accessed 2026-06-06 (2026)